

### Ejercicio 1 (2,5P)

Se pretende crear un **objeto literal** llamado “XHR” para realizar peticiones Ajax al servidor:

Se pide:

- a) Crear el objeto literal XHR con los siguientes métodos: (1,5P)
  - a. **get**( url, params, callbackFn)
  - b. **post**(url , data, callbackFn)
  - c. **serverRequest**( typeofRequest , url , params, callbackFn)
  - d. Crear los métodos anteriores. El método serverRequest sirve de apoyo al resto de métodos. Los parámetros de los métodos son descriptivos de lo que aceptan.
- b) Crear los métodos de forma que sean crossbrowser (IE8 y anteriores y Mozilla Firefox). (0,5P)
- c) Usar el objeto literal anterior para realizar una petición Ajax a la siguiente api rest mediante el método get. (0,5P)
  - a. <https://v2.jokeapi.dev/joke/Any>
  - b. **Mostrar por pantalla dicho chiste en un div con id="jokeDiv":**
    - i. **Se obtiene un objeto json del servidor del tipo:**

```
1. {
    "error": false,
    "category": "Programming",
    "type": "single",
    "joke": "A man is smoking a cig
on the cigarette pack? Smoking is h
errors.\\"",
    "flags": {
        "nsfw": false,
        "religious": false,
        "political": false,
        "racist": false,
        "sexist": false,
        "explicit": false
    },
    "id": 38,
    "safe": true,
    "lang": "en"
}
```

## Ejercicio 2 (2,5P) (Personajes de Rick and Morty)

### Personajes de Rick and Morty



**Rick Sanchez**

Especie: Human

Estado: Alive



**Morty Smith**

Especie: Human

Estado: Alive



**Summer Smith**

Especie: Human

Estado: Alive



**Beth Smith**

Especie: Human

Estado: Alive



**Jerry Smith**

Especie: Human

Estado: Alive



**Abadango Cluster Princess**

Especie: Alien

Estado: Alive

Se pretende crear un aplicación para mostrar los personajes de Rick and Morty a través de llamadas a su api de personajes (characters).

<https://rickandmortyapi.com/api/character> ( petición GET)

Un personaje de rick and morty se define por la siguiente estructura JSON definida en el archivo js/carácter\_rm.json que se te entrega.

Se pedirá mostrar los personajes de la primera página de personajes.

Cada página de personajes se solicita mediante la siguiente petición get a la url siguiente:

<https://rickandmortyapi.com/api/character/?page=2>

Donde page indica el número de página. La primera página no hay que especificar la página, sólo para la segunda página y sucesivas.

Se pide:

- Crear una función llamada “pedirPaginaPersonajesRM(pagina, callbackFn)” que acepta el número de página. Realizar una petición get a la url :  
<https://rickandmortyapi.com/api/character> para obtener las páginas y mostrar los personajes. Se mostrará la primera página de personajes nada más abrir dicha página index.html. Realizar la petición mediante el método fetch de la api de HTML5. (1P)
- Crear una función para mostrar los personajes como cards de bootstrap (se te da el código de un card ) llamada

muestraCardPersonaje( personajeData) que recibe los datos de un personaje. Los datos a mostrar del personaje serán name, species, status e image. Los datos recibidos son un objeto json con dichas propiedades. (1P)

- c) Crear una función llamada showMoreCharacters(pageNum) que será ejecutada cuando se presione el botón con id="btnMore" y que mostrará una nueva página de personajes borrando la que hay. Se encarga de hacer una nueva llamada a la api para traerse otra página de personajes de forma incremental. Cada vez que se pulse el botón se mostrará una nueva página de personajes (es la siguiente a la que estamos - atributo next de la figura de abajo). (0,5P)

Se muestra la estructura de información de los personajes de rick and morty.

## Get all characters

You can access the list of characters by using the `/character` endpoint.

```
GET https://rickandmortyapi.com/api/character
```

```
{
  "info": {
    "count": 826,
    "pages": 42,
    "next": "https://rickandmortyapi.com/api/character/?page=2",
    "prev": null
  },
  "results": [
    {
      "id": 1,
      "name": "Rick Sanchez",
      "status": "Alive",
      "species": "Human",
      "type": "",
      "gender": "Male",
      "origin": {
        "name": "Earth",
        "url": "https://rickandmortyapi.com/api/location/1"
      },
      "location": {
        "name": "Earth",
        "url": "https://rickandmortyapi.com/api/location/20"
      },
      "image": "https://rickandmortyapi.com/api/character/avatar/1.jpeg",
      "episode": [
```

Ejercicio 3. FakeStoreAPI. (5P)

Tenemos una api que muestra los detalles de carros de compra en comercios online (FakeStoreAPI).  
Se muestra un carro de compra, donde si se hace click en alguna línea de la izquierda se muestra en la parte derecha la información de dicho carro de compra con sus productos.

Lista de Carros

| ID | Usuario ID | Fecha                    |
|----|------------|--------------------------|
| 1  | 1          | 2020-03-02T00:00:00.000Z |
| 2  | 1          | 2020-01-02T00:00:00.000Z |
| 3  | 2          | 2020-03-01T00:00:00.000Z |
| 4  | 3          | 2020-01-01T00:00:00.000Z |
| 5  | 3          | 2020-03-01T00:00:00.000Z |
| 6  | 4          | 2020-03-01T00:00:00.000Z |
| 7  | 8          | 2020-03-01T00:00:00.000Z |

Detalles del Carro

ID: 1

Usuario ID: 1

Fecha: 2020-03-02T00:00:00.000Z

Productos:

- Producto ID: 1, Cantidad: 4
- Producto ID: 2, Cantidad: 1
- Producto ID: 3, Cantidad: 6

Pero queremos que la parte de la derecha al hacer click en el carro se vea de la siguiente forma: (suponiendo que se ha hecho click en el carro con id 1 )

Detalles del Carro (ID: 1)

| ID del Producto | Cantidad | Título                                                | Imagen                                                                                | Precio   |
|-----------------|----------|-------------------------------------------------------|---------------------------------------------------------------------------------------|----------|
| 1               | 4        | Fjallraven - Foldsack No. 1 Backpack, Fits 15 Laptops |  | \$109.95 |
| 3               | 6        | Mens Cotton Jacket                                    |  | \$55.99  |
| 2               | 1        | Mens Casual Premium Slim Fit T-Shirts                 |  | \$22.3   |

Queremos que al hacer click en el carro, se muestre la información de dicho carro mediante una petición get Ajax al servidor.

Se muestra una imagen de la petición a la api para traer información

Petición GET al endpoint <https://fakestoreapi.com/carts>

Se obtiene un objeto json con los datos de cada carro.

The screenshot shows a REST client interface with a dark theme. At the top, a green button labeled 'GET' is next to the endpoint '/carts'. Below this, a section titled 'Request samples' contains several tabs: 'JavaScript (Browser)' (selected), 'Node.js (Axios)', 'PHP', 'cURL', and 'Python'. The JavaScript tab displays the following code: 

```
fetch("https://fakestoreapi.com/carts")
  .then(response => response.json())
  .then(data => console.log(data));
```

 A 'Copy' button is to the right of the code. Below the request samples, a section titled 'Response samples' shows a status code of '200' in a green box. Underneath, the 'Content type' is listed as 'application/json'. To the right of the response content are buttons for 'Copy', 'Expand all', and 'Collapse all'. The response body is a JSON array containing one object, which is partially expanded to show its structure: 

```
[
  - {
    "id": 0,
    "userId": 0,
    - "products": [
      - {
        "id": 0,
        "title": "string",
        "price": 0.1,
        "description": "string",
        "category": "string",
        "image": "http://example.com"
      }
    ]
  }
]
```

Para obtener los datos de un carro.

Petición GET al id del carro al endpoint <https://fakestoreapi.com/carts/{:id}>

Donde {:id} representa un id de un carro. (quitar los dos puntos)

The screenshot shows a REST client interface with a dark theme. At the top, a green button labeled 'GET' is next to the path '/carts'. Below this, a section titled 'Request samples' contains five buttons: 'JavaScript (Browser)' (highlighted), 'Node.js (Axios)', 'PHP', 'cURL', and 'Python'. The JavaScript sample shows a fetch call to 'https://fakestoreapi.com/carts'. A 'Copy' button is to the right. Below the request samples is a section titled 'Response samples' with a green button labeled '200'. Underneath, a box shows the 'Content type' as 'application/json'. At the bottom, the JSON response is displayed with a 'Copy' button, 'Expand all' link, and 'Collapse all' link. The JSON is an array with one object containing 'id', 'userId', and 'products'.

```
GET /carts
```

Request samples

JavaScript (Browser) Node.js (Axios) PHP cURL Python

Copy

```
fetch("https://fakestoreapi.com/carts")
  .then(response => response.json())
  .then(data => console.log(data));
```

Response samples

200

Content type

application/json

Copy Expand all Collapse all

```
[
  - {
    "id": 0,
    "userId": 0,
    - "products": [
      + { ... }
    ]
  }
]
```

Llamada para obtener un producto por su id.

GET

/products/{id}

▼

Request samples

JavaScript (Browser)

Node.js (Axios)

PHP

cURL

Python

Copy

```
fetch('https://fakestoreapi.com/products/1')
  .then(response => response.json())
  .then(data => console.log(data));
```

Response samples

200

Content type

application/json

Copy

```
{
  "id": 0,
  "title": "string",
  "price": 0.1,
  "description": "string",
  "category": "string",
  "image": "http://example.com"
}
```

Se pide:

- a) (2P) Crear una función llamada `peticionesCarros(url, callbackFn)` mediante el objeto `XMLHttpRequest` que obtiene la información de todos los carros (llamando a la api mediante `get` a la siguiente url:

<https://fakestoreapi.com/api/carts>

- a. La función de callback llamará a la función para mostrar la tabla con las lista de carros.  
b. De cada carro se muestra la siguiente información:

- i. Id del carro
- ii. Id del usuario
- iii. Fecha de compra
- iv. Array de products : [  
    {  
        "productId" : 1,  
        "quantity": 4  
    }

- b) Crear la función `displayCartDetails(cart)` que muestra los detalles de un carro que recibe. Esta función tendrá que llamar a la siguiente para mostrar los detalles de un producto. No tenemos el producto, sólo su id. Rellenar las tablas con los datos de los carros. (1,5P)

- c) Crear la función `fetchProductDetail(productId, quantity)` que realiza una petición del producto por id y su cantidad y agrega una fila a la tabla de detalles del carro. Se hará una petición por `get` al siguiente endpoint para obtener información de un producto Rellenar la tabla de detalles del carro según la figura adjunta del ejercicio (1,5P)

- a. <https://fakeapistore.com/products/{idProducto}>

Nota: Se suministra la plantilla con estilos `css` y `html` para las tablas correspondientes a la lista de carros y los detalles de un carro.

Leer los archivos `products.json` y `carts.json` que se entregan para entender la información que llega de las peticiones `Ajax` al servidor.